

File Backup and Monitoring System

A Client–Server Architecture for Automated, Configurable File Backup

Fares Ben Hadj Slimane

Taysir

Internship Report

30-Day Project

August 12, 2026

Contents

1	Introduction	2
1.1	Context and Motivation	2
1.2	Problem Statement	2
1.3	Project Evolution	2
1.4	Objectives	2
1.5	Report Structure	2
2	System Architecture	3
2.1	Overview	3
2.2	Technology Stack	3
2.3	Design Rationale	3
3	Client Agent	4
3.1	Responsibilities	4
3.2	File System Monitoring	4
3.3	Local Persistence	4
3.4	Two-Phase Upload Cycle	5
3.5	Negotiation Payload Format	5
4	Central Server	6
4.1	Responsibilities	6
4.2	Server-Side File Storage Layout	6
4.3	Security: Path and Ownership Validation	6
4.4	Deduplication and Hash Negotiation	7
4.5	Configuration Update Strategy	7
5	Database Design	8
5.1	Server-Side Schema (MySQL)	8
5.1.1	User Table	8
5.1.2	Device Table	8
5.1.3	Watched Root Table	8
5.1.4	Watched Extension Table	8
5.1.5	Folder Table	8
5.1.6	File Table	9
6	Administration Panel	10
6.1	Authentication	10
6.2	Features	10
6.3	Indicative API Surface	10
7	Security Considerations	11

8	Project Planning	12
8.1	Timeline	12
9	Conclusion	13
9.1	Future Work	13

Chapter 1

Introduction

1.1 Context and Motivation

1.2 Problem Statement

Organizations need a reliable way to protect important files on employee workstations against accidental loss or modification, while retaining visibility into what changes occur and where. Manual backups do not scale across multiple machines and multiple users per machine, which motivates an automated, centralized, and configurable backup solution.

1.3 Project Evolution

The project was initially scoped as a pure file-change logging system, recording metadata about file creations, modifications, and deletions without handling file content. Following clarification with the project's supervisor, the scope was refined to a full **file backup system**: the agent not only detects changes but uploads the actual new content of modified files to a central server, which maintains an up-to-date mirrored copy of each tracked file.

1.4 Objectives

- Monitor, on each client workstation, only the folders and file extensions explicitly designated by the administrator.
- Detect file changes in real time and periodically upload new versions of modified files to a central server.
- Avoid redundant uploads by negotiating file identity via content hashes before transferring any file content.
- Reproduce, on the server, a filesystem hierarchy mirroring each device and each user of that device.
- Provide an administration panel to browse users, devices, upload history, and folder structure, and to remotely reconfigure the upload period, watched folder roots, and watched file extensions on a per-user basis.

1.5 Report Structure

Chapter 2

System Architecture

2.1 Overview

The system is composed of three logical components:

1. **Client Agent** – a background application installed on each monitored workstation, responsible for detecting changes to configured files and uploading new content periodically. Written in Python.
2. **Central Server** – a single Python (FastAPI) Web API application exposing endpoints for agents to negotiate and upload file content, and endpoints for the administration panel.
3. **Administration Panel** – a JWT-authenticated web interface allowing an administrator to consult activity, statistics, folder structure, and to adjust per-user configuration.

2.2 Technology Stack

Component	Technology
Client Agent	Python 3, <code>watchdog</code> library
Client local storage	SQLite (flagged-changes buffer)
Server	Python, FastAPI
Server ORM	SQLAlchemy
Server database	MySQL
Server file storage	Local filesystem, mirrored per device/user
Admin authentication	JWT (JSON Web Tokens)
Client-server transport	HTTP / JSON (negotiation) + multipart (upload)

2.3 Design Rationale

Python was adopted for both the client and the server. On the client side, this was chosen for its strong file-system-monitoring ecosystem (`watchdog`) and cross-platform portability. On the server side, the initial plan used ASP.NET Core; this was revised to Python following a requirement from the project’s supervisor, so that the software department maintaining the project going forward – who are proficient in Python but not in .NET – can support it.

Chapter 3

Client Agent

3.1 Responsibilities

- Monitor only the folder roots and file extensions configured by the administrator for the current user.
- Detect file creation and modification events in real time.
- Maintain a local record of files flagged as changed, until their upload is confirmed by the server.
- Periodically negotiate with the server (via content hashes) which flagged files actually require upload, then transmit their content.
- Apply configuration updates (period, watched roots, watched extensions) received from the server.

3.2 File System Monitoring

File system events are captured using the Python `watchdog` library, which wraps OS-native notification mechanisms (`inotify` on Linux, `ReadDirectoryChangesW` on Windows) behind a single cross-platform API. Only paths under the configured watched roots, and matching a configured extension, are considered; all other events are ignored at the handler level.

3.3 Local Persistence

Detected changes are recorded in a local SQLite database rather than being uploaded immediately:

Column	Type	Description
id	INTEGER (PK)	Local identifier, unique per agent
path	TEXT	Absolute path of the affected file
last_known_hash	TEXT	Hash of the last version confirmed uploaded
flagged	BOOLEAN	Whether the file has unconfirmed local changes

A file's `flagged` status is only cleared once the server has confirmed successful receipt of the corresponding upload – never on the basis of a negotiation response alone (see Chapter 4), since the file may change again during the round-trip to the server.

3.4 Two-Phase Upload Cycle

At the end of each configured period, the agent performs two requests:

1. **Negotiation request** – sends the path and current content hash of every flagged file, together with the device/user identifier.
2. **Upload request** – sends the full content, via multipart form data, of only the files the server indicated it actually needs (i.e. hash mismatch or unknown file).

Files the server reports as not needing upload remain flagged locally and are re-evaluated at the next cycle, guaranteeing no change is silently lost due to a stale negotiation response.

3.5 Negotiation Payload Format

```
{
  "device_id": "string",
  "user": "string",
  "files": [
    {"path": "string", "hash": "sha256..."}
  ]
}
```

Server response:

```
{
  "files_to_upload": ["path1", "path3"],
  "period": "integer (seconds)",
  "watched_roots": ["string", "..."],
  "watched_extensions": ["string", "..."]
}
```

Chapter 4

Central Server

4.1 Responsibilities

- Expose a negotiation endpoint that compares submitted hashes against stored ones and returns the files actually requiring upload.
- Expose an upload endpoint accepting file content and writing it to the correct location in the server's mirrored filesystem.
- Reconstruct, and persist in the database, the folder tree corresponding to each uploaded file's path.
- Return each user's current configuration (period, watched roots, watched extensions) with every negotiation response.
- Expose authenticated endpoints for the administration panel.

4.2 Server-Side File Storage Layout

The server maintains a filesystem hierarchy rooted at a dedicated backup directory, structured as:

```
backup_root/ {device_id}/ {user}/ <mirrored original path>
```

Uploading a new version of a file replaces the previous version in place; no version history is currently retained (see Section 9.1).

4.3 Security: Path and Ownership Validation

Because the destination path for an uploaded file is derived from client-supplied data (device id, user, and original file path), two complementary safeguards are applied:

1. **Restricted service root** – the server process only ever operates within the dedicated backup root directory, limiting the impact of any validation failure.
2. **Path containment check** – before any write, the fully resolved destination path is verified to remain a descendant of that specific device/user's folder; any path attempting to escape it (e.g. via `..` segments) is rejected.

This defense-in-depth approach also addresses the multi-user-per-device case: even though the client-side agent may run with elevated privileges to read across multiple OS users' files, the server does not inherently know which OS user a given upload belongs to and must not trust this claim blindly – the same path-containment check prevents one user's uploaded content from ever being written into another user's folder on the same device.

4.4 Deduplication and Hash Negotiation

See Chapter 3 for the client-side flow. On the server side, each tracked file's `content_hash` is stored alongside its metadata; a negotiation request is answered by comparing submitted hashes against these stored values, so that file content is only transferred over the network when it has genuinely changed.

4.5 Configuration Update Strategy

As established early in the project's design, configuration changes (originally just the reporting period, later extended to watched roots and watched extensions) are delivered to the agent via a **pull-based** mechanism: the administrator's changes are persisted in the database, and returned to the agent as part of the response to its own periodic negotiation request. A push-based alternative (server-initiated connection to the agent) was considered and rejected early in the design process, due to NAT/firewall traversal issues, the need to track agent IP addresses (which are not guaranteed stable), and the increased attack surface of requiring an open listening port on every monitored machine.

Chapter 5

Database Design

5.1 Server-Side Schema (MySQL)

5.1.1 User Table

Column	Type	Description
id	INT (PK)	Unique identifier
name	VARCHAR	User identifier
device_id	INT (FK)	Device this user belongs to
period	INT	Configured upload period (seconds), per user

5.1.2 Device Table

Column	Type	Description
id	INT (PK)	Unique identifier
name	VARCHAR	Device / hostname identifier

5.1.3 Watched Root Table

Column	Type	Description
id	INT (PK)	Unique identifier
user_id	INT (FK)	Owning user
path	VARCHAR	Folder root considered for backup

5.1.4 Watched Extension Table

Column	Type	Description
id	INT (PK)	Unique identifier
user_id	INT (FK)	Owning user
extension	VARCHAR	File extension considered for backup

5.1.5 Folder Table

A self-referencing (adjacency-list) structure represents each user's folder tree, used by the admin panel to display structure without exposing file contents.

Column	Type	Description
id	INT (PK)	Unique identifier
name	VARCHAR	Folder name
parent_id	INT (FK, nullable)	Parent folder (NULL = root)
user_id	INT (FK)	Owning user

5.1.6 File Table

Column	Type	Description
id	INT (PK)	Unique identifier
folder_id	INT (FK)	Containing folder
name	VARCHAR	File name
content_hash	VARCHAR	Hash of the currently stored version
last_updated	DATETIME	Timestamp of the last accepted upload

Chapter 6

Administration Panel

6.1 Authentication

Access to the administration panel is protected using JWT (JSON Web Tokens). Administrator credentials are submitted to a dedicated login endpoint, which returns a signed token upon success; this token is required in the **Authorization** header of every subsequent request to a protected endpoint. Agent-facing endpoints (negotiation, upload) are intentionally kept outside this authentication scheme, as agents are not administrators – establishing two distinct trust boundaries within the same service.

6.2 Features

- List of devices and users, with their current configuration.
- Upload history per user.
- Read-only folder tree view per user (structure only – no file content access from the panel).
- Statistics and charts (e.g. uploads over time, most active users/devices, storage usage).
- Per-user configuration management: upload period, watched folder roots, and watched file extensions – all propagated to the agent via the pull-based mechanism described in Chapter 4.

6.3 Indicative API Surface

Endpoint	Purpose
POST /api/auth/login	Authenticate admin, issue JWT
GET /api/users	List users and devices
GET /api/users/{id}/uploads	Upload history for a user
GET /api/users/{id}/tree	Folder tree for a user
GET /api/stats	Aggregate statistics for dashboard charts
PUT /api/users/{id}/period	Update upload period
POST/DELETE /api/users/{id}/roots	Manage watched roots
POST/DELETE /api/users/{id}/extensions	Manage watched extensions

Chapter 7

Security Considerations

- Transport should be secured with HTTPS in production, given that actual file content is transmitted, not just metadata.
- Administration endpoints are protected via JWT; agent endpoints use a lighter mechanism appropriate to their trust level.
- Server-side path containment checks and a restricted service root prevent path traversal and cross-user file writes (see Chapter 4).
- No IP addresses are stored, reducing exposure in case of a database compromise, and removing the need to track agent reachability at all.

Chapter 8

Project Planning

8.1 Timeline

Days	Milestone
1–3	Research, architecture design, database schema, API contract
4–10	Client agent: file watching with <code>watchdog</code> , watched-root/extension filtering, local SQLite flagging
11–14	Client–server negotiation and upload cycle (hash comparison, multipart transfer)
15–19	Server: negotiation and upload endpoints, folder-tree reconstruction, path/ownership validation, MySQL persistence
20–25	Admin panel: authentication, user/device listing, upload history, folder tree view, statistics
26–27	Per-user configuration management (period, roots, extensions) via the admin panel
28–30	Testing, edge cases, documentation, final report

Chapter 9

Conclusion

9.1 Future Work

The following features were identified during design discussions as valuable extensions, but are out of scope for the current 30-day timeline:

- Administrator ability to delete untracked or orphaned folders on the server.
- A time-limited (e.g. 30-day) retention window for files deleted on the client, rather than immediate/permanent removal.
- Version history for backed-up files, rather than in-place replacement of the latest version.
- Packaging the client agent as a standalone executable (e.g. via PyInstaller) to remove the dependency on a pre-installed Python interpreter on target machines.